

Module 2: Search Algorithms & Game Playing

Uninformed Search (BFS/DFS/UCS/IDS), Heuristic A*, Minimax & Alpha-Beta Pruning

Module II: Problem Solving by Search Agents — Complete 6-Topic Syllabus Tracker

Topic 8: Search-Based Problem Solving (5-Tuple Formal Problem Formulation)

Topic 11: Heuristic Search (Greedy Best-First, A* Search & Admissibility)

Topic 13: Search in Complex Environments (Belief States & Contingencies)

Topic 9 & 10: State-Space Search (BFS, DFS, UCS, DLS, IDDFS & Graph Search)

Topic 12: Local Search (Hill Climbing, Simulated Annealing & Genetic Algo)

Topic 14: Adversarial Game-Tree Search (Minimax Algorithm & Alpha-Beta Pruning)

Topic 8: Search Problem Formulation (The 5-Tuple)

Formal Problem Formulation 5-Tuple: $\langle S_0, A(s), \text{Result}(s, a), \text{GoalTest}(s), c(s, a, s') \rangle$

- Initial State (S_0):** The starting state the agent begins in (e.g., $\text{In}(\text{Arad})$).
- Actions ($A(s)$):** Set of legal executable actions available in state s .
- Transition Model ($\text{Result}(s, a)$):** Returns the successor state after taking action a in state s .
- Goal Test ($\text{GoalTest}(s)$):** Boolean function determining if state s is a target goal.
- Path Cost Function ($c(s, a, s')$):** Step cost $g(n)$ of taking action a to reach s' . Total path cost = $\sum c$.

Topic 9 & 10: Uninformed (Blind) Search Strategies

Algorithm	Frontier Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
1. BFS	FIFO Queue	$O(b^{d+1})$	$O(b^{d+1})$ (High)	Yes	Yes (if equal step costs)
2. DFS	LIFO Stack / Recursion	$O(b^m)$	$O(b \cdot m)$ (Low)	No (in infinite trees)	No
3. UCS	Priority Queue by $g(n)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	Yes	Yes (for step costs $\geq \epsilon > 0$)
4. IDDFS	Iterative LIFO Stack	$O(b^d)$	$O(b \cdot d)$ (Low)	Yes	Yes (if equal step costs)

Topic 11: Informed Heuristic Search & The A* Algorithm

1. Evaluation Functions: - **Greedy Best-First Search:** $f(n) = h(n)$ (Fast, but non-optimal; can get trapped in loops). - **A* Search:** $f(n) = g(n) + h(n)$ where $g(n)$ is the exact cost from start to n , and $h(n)$ is estimated cost to goal.

2. Admissibility and Consistency Conditions for A* Optimality: - **Admissible Heuristic:** $h(n) \leq h^*(n)$ for all n (never overestimates the true remaining cost). Guarantees A* is optimal in Tree Search! - **Consistent (Monotonic) Heuristic:** $h(n) \leq c(n, a, n') + h(n')$ (satisfies the triangle inequality). Guarantees A* is optimal in Graph Search with zero node re-expansions!

Topic 12: Local Search & Optimization Algorithms

Local Search Algorithm	Operating Strategy & State Space Navigation	Key Pitfall & Solution
1. Hill Climbing	Greedy local search that continuously moves in the direction of increasing elevation (value).	Gets trapped on Local Maxima, Plateaus, and Ridges . Solution: Random-restart hill climbing.
2. Simulated Annealing	Escapes local maxima by allowing downhill moves with probability $P = e^{\frac{\Delta E}{T}}$. Temperature T decreases according to a cooling schedule.	Guaranteed asymptotic convergence to global optimum as $T \rightarrow 0$ if cooled sufficiently slowly.
3. Genetic Algorithms	Population-based search simulating natural selection: Selection (fitness proportionate) $\rightarrow$ Crossover $\rightarrow$ Mutation .	Excellent for massive, complex, multi-modal search landscapes (e.g., TSP, circuit design).

Topic 14: Adversarial Game-Tree Search (Minimax & Alpha-Beta Pruning)

1. Minimax Value Recursive Definition:

$$\text{Minimax}(s) = \begin{cases} \text{Utility}(s) & \text{if Terminal}(s) \\ \max_{a \in A(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in A(s)} \text{Minimax}(\text{Result}(s, a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$

Alpha-Beta ($\alpha - \beta$) Pruning Invariant Rule

Maintain two bounds during recursive DFS traversal:

- α = the highest value choice found so far along the path for **MAX** (initialized to $-\infty$).
- β = the lowest value choice found so far along the path for **MIN** (initialized to $+\infty$).

Pruning Condition: Whenever $\alpha \geq \beta$, prune the remaining subtree below the current node immediately! (Reduces effective branching factor from b to $\sqrt{b}$, doubling search depth!).

M2 Active Recall & Exam Questions

Q1. Prove that A* Search is optimal when using an Admissible Heuristic in Tree Search. (8 Marks)

Let G_2 be a suboptimal goal with cost $g(G_2) > C^*$, and G be an optimal goal with $g(G) = C^*$.

Suppose G_2 is in the frontier and is about to be selected. For any unexpanded node n on the optimal path to G :

- Since h is admissible: $f(n) = g(n) + h(n) \leq C^*$.
- For the suboptimal goal: $f(G_2) = g(G_2) + h(G_2) = g(G_2) > C^*$.
- Therefore, $f(n) \leq C^* < f(G_2) \implies f(n) < f(G_2)$.

Hence, A* will always expand node n on the optimal path before ever selecting G_2 , guaranteeing that G_2 can never be chosen before an optimal path is found!